

Report HMW1

Filippo Paris (270270) Giorgio Mattina (984347)
filippo.paris@mail.polimi.it giorgio.mattina@mail.polimi.it

April 2025

1 Introduction

Linear Predictive Coding (LPC) is a widely used method in speech processing that basically models speech signals by predicting the current sample based on a linear combination of past samples. LPC allows for efficient encoding and synthesis of speech by reducing the amount of data needed to represent the original signal. The method first involves estimating a set of coefficients (LPC coefficients) that describe the spectral envelope of the signal. These coefficients are then used to reconstruct the speech signal with minimal loss in quality.

For these reasons, LPC has been widely applied in speech synthesis systems, including early educational devices such as the Speak and Spell developed by Texas Instruments in the 1970s, which is the purpose of our assignment. The Speak and Spell utilized LPC-based synthesis to generate intelligible speech from compressed representations of words, making it one of the first consumer products to use digital speech synthesis.

In this project, we aim to implement in Matlab the LPC-10 algorithm: the task involves extracting LPC coefficients from speech frames, synthesizing the speech signal from these parameters, and evaluating the quality of the reconstructed signal. Note that we won't report all the code directly in this PDF, as it is already present in the Matlab file delivered.

2 Encoder.m

Initially, we load the input audio file (e.g. 'ces.wav') using `audioread()`. The sample rate of the input signal is checked and resampled to 8kHz, if necessary. This is because LPC-10 operates at an 8kHz sample rate, and using a higher sample rate could unnecessarily increase computation complexity. So, we're resampling to 8kHz to ensure that the signal matches the intended processing rate of the LPC model.

Normalizing the signal ensures then that the amplitude of the audio signal is within a standard range. In this case, it's scaled so that the peak value is 1. This step is crucial in order to prevent issues with varying signal amplitudes, which could lead to instability or biased analysis during the LPC calculation.

```

1 s = s / max(abs(s));
2 b = [1, -0.975];
3 s = filter(b, 1, s);

```

The Pre-Emphasis filter is a high-pass filter that boosts the high-frequency components of the signal. This is done to compensate for the fact that speech tends to have more energy in the lower frequencies, and boosting the higher frequencies improves the signal's clarity, especially for high-pitched sounds (like for example consonants). The filter is here implemented with a coefficient of -0.975 to apply a gentle emphasis to the higher frequencies. Once done that, the speech signal is divided into overlapping frames for analysis.

We define the STFT parameters:

winLen = 256 : this sets the frame length to 256 samples, which has been chosen as a reasonable balance between time and frequency resolution.

hopSize = 128 : this is the step size between successive frames. With a hop size of 128, there is a 50 percent overlap between consecutive frames. This overlap is commonly used to maintain continuity between frames and reduce distortion.

win = hamming(winLen) : a Hamming window is applied to each frame to reduce spectral leakage. The Hamming window is useful because it tapers the signal at the frame boundaries, helping to minimize edge effects in the frequency domain.

nFrame : the number of frames is computed based on the signal length, frame length, and hop size.

The next 'for loop' segments the speech signal into overlapping frames. Each frame is multiplied by the Hamming window (win) to minimize the discontinuities at the frame boundaries. We use overlapping frames to ensure that the speech signal is represented smoothly across frames, avoiding abrupt transitions between adjacent frames. The Hamming window is also necessary in order to reduce spectral leakage during the Fourier Transform and smooth out the signal, especially at the beginning and at the end of the frame.

2.1 LPC-10 Parameters Initialization

We also define:

pitch : an array to store pitch estimates for each frame. This is only relevant for voiced frames (vowels and other periodic sounds).

gains : an array to store the gain values for each frame, representing the energy of the signal.

power : where we store the computed power of each frame, which is used for gain computation.

2.2 LpFilt definition

`lpFilt` is a low-pass filter designed using the `designfilt` function, which is applied to the prediction error signal in the LPC process. The purpose of this filter is to remove high-frequency components from the prediction error signal, especially for voiced speech frames where the prediction error can be dominated by high-frequency noise. By reducing this high-frequency content, the filter smooths out the error signal, making pitch detection more accurate and preventing the analysis from being affected by noise.

We define the filter with the following parameters:

- **PassbandFrequency:** 700 Hz, which specifies the cutoff frequency below which the signal will pass without attenuation.
- **StopbandFrequency:** 900 Hz, which specifies the frequency above which the signal will be attenuated.
- **PassbandRipple:** 1 dB, which controls how much ripple (variation) is allowed in the passband.
- **StopbandAttenuation:** 60 dB, which determines how much attenuation is applied to frequencies in the stopband.
- **SampleRate:** `fs`, which is the sampling rate of the audio signal (8 kHz in this case).

This low-pass filter is applied to the prediction error signal `e` for voiced speech frames, after which the pitch detection is performed using the AMDF (Average Magnitude Difference Function) method. The result is a cleaner error signal that enhances the accuracy of pitch estimation for voiced sounds.

If the `voicedIdx(ii)` flag is set to 1 (indicating that the current frame is voiced), the prediction error signal `e` is passed through the low-pass filter `lpFilt` to reduce high-frequency noise before pitch estimation.

The flag `doPlot` controls instead whether to visualize the results. When set to false, no plots are generated. It's useful for debugging or visualization but typically not needed for the final implementation

Then the function `voicedframedetection` identifies audio frames based on the zero-crossing rate (ZCR), a method used to distinguish periodic (voiced) sounds from aperiodic (unvoiced) ones. It takes three input arguments: `'s'` (the signal), `'win'` (the window function), and `'hopsize'` (the step between frames). The function calculates the ZCR for each frame by counting the number of sign changes between consecutive samples, which is stored in the `'zc'` array.

Frames with a high ZCR are considered unvoiced, while those with a low ZCR are classified as voiced. Additionally, frames with very low signal energy are classified as unvoiced to handle silent or low-energy sections. The function outputs two arrays: `'voicedIdx'`, indicating whether a frame is voiced (1) or

unvoiced (0), and 'zc', containing the ZCR values for each frame. This method efficiently distinguishes voiced from unvoiced speech components, aiding in further speech processing tasks like pitch detection. The choice of ZCR and energy thresholds ensures accurate classification, even in noisy conditions.

2.3 Estimate LPC filters

Finally, we enter in the LPC Coefficients Estimation. For each frame `ii`, the signal `sn` is either voiced or unvoiced, as detected previously by `voicedIdx`. The order of LPC model is set to 10 for voiced frames and 4 for unvoiced frames. This choice reflects instructions we were given.

The process starts by iterating over all previously segmented and windowed frames of the audio signal. For each frame, the code extracts the signal vector `sn` representing the current short-time segment of the pre-processed speech waveform. Since speech is considered quasi-stationary over short time periods, this frame-based analysis is essential to accurately model its characteristics.

To estimate the LPC coefficients, the autocorrelation of the current frame is computed using the `xcorr` function. This function returns a symmetric autocorrelation vector centered around zero lag. However, for LPC analysis, we need only the non-negative lags (from lag 0 onwards). The center of the autocorrelation vector is therefore computed, and the positive-lag portion is extracted starting from that central index.

An important design decision is the choice of the LPC model order, which is adaptively determined based on the voicing state of the frame. If the frame is classified as voiced, a higher-order model is used: specifically, an LPC order of 10. For unvoiced frames, a lower order of 4 suffices. This adaptive approach optimizes the model complexity according to the spectral richness of the speech segment.

The actual LPC coefficients are estimated using the `levinson` function, which implements the Levinson-Durbin recursion to efficiently solve the Toeplitz system derived from the autocorrelation method. The result is a set of filter coefficients `a{ii}` for the current frame. These coefficients define the all-pole filter that models the vocal tract for that segment. An alternative method using the built-in `lpc` function is commented out in the code but here explained; both approaches are valid, but the explicit use of `levinson` provides more transparency and control.

To safeguard against numerical issues, the code includes a check for very low-energy frames. If the frame's energy is negligible, a trivial filter (with only a coefficient of 1 and zeros elsewhere) is assigned to prevent undefined behavior or the propagation of NaNs.

```

1 if sum(sn.^2) < 1e-6
2     a{ii} = [1, zeros(1, orderLPC)];
3     e = sn;
4 end

```

In some cases in fact, especially with very small or near-zero signal values, certain mathematical operations (like computing LPC coefficients or performing filtering) can produce undefined or infinite results. This could happen if the signal contains zeros or very small numbers that lead to division by zero errors or other numerical issues. Avoiding NaN values during the LPC analysis process doesn't significantly alter the signal since the value of epsilon that we've chosen is very small. In fact, the choice of 1e-6 is a typical value that's small enough to not noticeably affect the signal but large enough to prevent numerical issues like dividing by zero or encountering very small numbers that could cause inaccuracies during subsequent steps.

As we said before, the `lpc()` function is another way of computing the LPC coefficients for each frame. These coefficients represent the filter that models the speech signal.

```
1 a{ii} = lpc(sn .* win, orderLPC);  
2  
3 e = filter(a{ii}, 1, sn);
```

The speech frame `sn` is multiplied by a Hamming window (`win`) before computing the LPC coefficients. This windowing step is crucial for reducing the spectral leakage that occurs when analyzing a finite segment of a signal. The Hamming window is commonly used because it provides a good trade-off between time-domain localization and frequency-domain resolution.

The parameter `orderLPC` defines the order of the LPC model. The choice of order determines how many past samples of the signal are used to predict the current sample. For voiced speech (periodic), a higher order is used (10 in this case), while for unvoiced speech (aperiodic), a lower order (4) is typically used.

After computing the LPC coefficients for a frame, the prediction error `e` is calculated by filtering the original signal `sn` with the LPC filter. The Mean Squared Error (MSE) measures how well the LPC model approximates the original signal. A lower MSE indicates a better fit. This is useful for evaluating the quality of the LPC encoding.

The decision to use the Levinson-Durbin recursion over the standard LPC function in this code is primarily motivated by several practical considerations: one of the main reasons is that the Levinson-Durbin algorithm is specifically designed to efficiently solve the Toeplitz system of equations generated from the autocorrelation of the signal. Moreover, Levinson-Durbin not only computes the LPC coefficients but also directly provides the prediction error, which is crucial for subsequent stages of the analysis. This makes the code flow more seamless and efficient, as it eliminates the need for additional steps to calculate the residual, which would be necessary if the `lpc` function were used. Ultimately, the choice of Levinson-Durbin is driven by the need for a robust, efficient, and flexible method that can handle various speech conditions and provide accurate LPC coefficients along with the prediction error. This makes it a natural fit for speech analysis applications like the one in this code.

Then, for voiced frames, pitch detection is performed using the `pitchdetectionamdf()` function. The gain is also calculated based on the power of the error signal and pitch. This gain factor is used to adjust the amplitude of the predicted signal. For unvoiced frames, instead, the gain is computed using the standard deviation of the prediction error, reflecting the randomness of unvoiced speech sounds.

After processing all frames, the LPC coefficients (`a`), gains, pitch estimates, and other parameters are saved into a `.mat` file. This allows for efficient storage and future retrieval, such as for decoding or further analysis.

2.4 Setting `doPlot`: `True`

Setting `doPlot = true` will activate a series of visualizations and plots during each iteration of the LPC analysis. These plots help to better understand the performance of the LPC model, the spectrum of the signal, and the quality of the prediction error.

First, we plot the FFT (Fast Fourier Transform) of the original signal frame. The FFT gives us the frequency spectrum of the signal, showing how the signal is distributed across various frequencies. The amplitude is shown in decibels (dB). The LPC spectrum of the frame (`H`), which is the spectrum obtained from the LPC filter. The LPC model represents the signal's spectral envelope, and this is computed using the `freqz()` function based on the LPC coefficients (`aii`) obtained earlier. By comparing the FFT of the original signal and the LPC spectrum, we can visually assess how well the LPC model matches the original signal's frequency content. If the red LPC spectrum closely follows the blue FFT spectrum, it indicates that the LPC model is a good approximation of the signal's frequency characteristics.

The second subplot shows the prediction error `e` in the time domain. The prediction error is the difference between the original signal and the predicted signal based on the LPC coefficients. This error is plotted over time. The plot of the prediction error in the time domain helps to visualize how well the LPC model has been able to predict the speech signal. If the error is large or erratic, it indicates that the LPC model is not performing well for that frame, possibly due to a mismatch between the model and the signal's characteristics. If the prediction error is small and consistent, this means that the LPC model has successfully predicted most of the signal's characteristics, and the residual (error) is minimal.

Finally, the third subplot plots the frequency spectrum of the prediction error. Just like the original signal's spectrum, the FFT of the prediction error is plotted, but this time it reflects the frequency components of the error signal. The plot shows the magnitude of the prediction error in dB across the frequency axis. This plot provides insight into which frequencies contribute most to the prediction error. Large peaks in the error spectrum indicate frequencies where

the LPC model struggles to predict the signal accurately. The goal is for the prediction error spectrum to be as flat and low as possible across the frequency range, as this indicates that the LPC model is effectively capturing most of the signal's energy. If the prediction error has large spectral components at certain frequencies, it means those components were poorly predicted by the LPC model and are now residuals.

3 Decoder.m

In the LPC decoder, we reconstruct the original speech signal from the LPC parameters and pitch information. The reconstruction process is performed frame-by-frame, using an excitation signal (either voiced or unvoiced), an LPC synthesis filter, and overlap-add processing to concatenate frames. The final output is post-processed with a de-emphasis filter and played back. The script begins by loading the encoded LPC parameters from a `.mat` file. This file is expected to contain:

- `a`: a cell array of LPC coefficients for each frame
- `gain`: energy or gain values for each frame.
- `voicedIdx`: binary indicators for voiced/unvoiced classification.
- `pitch`: pitch periods for voiced frames.
- Other parameters like `fs` (sampling frequency), `winLen` (frame length), and `hopSize`

The script then enters a loop that iterates over each frame in the signal. The goal of this loop is to reconstruct each frame individually and add it to the final output waveform.

```

1  for ii = 1:nFrame
2  excite = generateexcitationsignal(voicedIdx(ii),
    ↪ gains(ii), pitch(ii), frameLen);
3  sRec_frame = filter(1, a{ii}, excite);
4
5  % Overlap-add
6  startIdx = (ii - 1) * hopSize + 1;
7  synthS(startIdx : startIdx + frameLen - 1) =
    ↪ synthS(startIdx : startIdx + frameLen - 1) +
    ↪ sRec_frame;
8  end

```

The function `generateexcitationsignal()` is responsible for generating the excitation signal, which serves as the input to the LPC synthesis filter. The type of excitation depends on the voicing classification of the frame:

- Voiced frames are modeled using a periodic impulse train, where impulses are spaced at intervals corresponding to the pitch period. This mimics the vibration of the vocal cords during voiced speech.
- Unvoiced frames are modeled using white Gaussian noise to simulate turbulent airflow

The excitation signal is also scaled by the frame's gain to preserve the amplitude characteristics of the original speech.

After generating the excitation signal, it is passed through an all-pole digital filter using the `filter()` function. The filter is defined by the LPC coefficients `aii`, which were originally computed during the encoding phase to represent the spectral envelope of the speech frame.

This process reconstructs an approximation of the original speech waveform based on the source-filter model:

$$\hat{s}(n) = \sum_{k=1}^p a_k \hat{s}(n-k) + e(n) \quad (1)$$

Where $\hat{s}(n)$ is the reconstructed speech sample, $a(k)$ are the LPC coefficients, $e(n)$ is the excitation signal.

3.1 Overlapp And Add

Once the speech for a frame is synthesized, it must be placed at the correct position in the output signal. This is achieved through an overlap-add approach: The start index of each frame is computed as $(\text{framenumbers} - 1) * \text{hopsizes} + 1$, then the synthesized frame is added to the corresponding segment in the `synthS` vector. If frames overlap, their values are added together.

3.2 De-emphasis Filtering

After reconstructing the full signal, a de-emphasis filter is applied. This is the inverse operation of the pre-emphasis filter used during encoding, which boosts high-frequency components to counteract the natural spectral tilt of human speech.

The de-emphasis filter has the form:

$$H(z) = \frac{1}{1 - 0.975z^{-1}} \quad (2)$$

This high-pass filter reduces the emphasis on higher frequencies and restores the natural spectral balance of the speech signal. It plays a crucial role in improving the perceptual quality of the synthesized speech.

4 PitchDetectionAMDF

Pitch detection is a fundamental task in speech and audio signal processing. The pitch (or fundamental frequency) of a voiced speech segment corresponds to the rate at which the vocal folds vibrate, and is crucial for encoding prosody, speaker identity, and phonetic content. Accurate pitch estimation is vital in a wide range of applications including speech synthesis, coding, recognition, and speaker analysis. The `pitchdetectionamdf` function is an implementation of a pitch estimation algorithm based on the **Average Magnitude Difference Function (AMDF)**, which is a time-domain method that compares the signal with time-shifted versions of itself.

The MATLAB function `pitchdetectionamdf` takes as an input a segment of speech signal `e` and outputs an estimated pitch period (in samples). The function assumes the input segment corresponds to a voiced frame and computes the pitch based on minimizing the AMDF over a reasonable range of pitch lags.

```
1 winLen = length(e);
2 len = length(e);
3 lags = winLen;
4 amd = zeros(1,lags);
5 e = [e; zeros(lags,1)];
6 for k=1:lags
7     for j=1:len
8         amd(k) = amd(k) + abs(e(j) - e(j + k));
9     end
10    amd(k) = amd(k) / len;
11 end
12 pitch = find(amd == min(amd(25:80)));
13 end
```

The AMDF algorithm computes a distance metric for each possible lag k , measuring how different the signal is from a time-shifted version of itself. Unlike autocorrelation, which uses multiplication, AMDF uses the absolute value of differences, which avoids the need for multipliers and makes the method computationally more efficient.

Mathematically the function corresponds to:

$$\text{AMDF}(k) = \frac{1}{N} \sum_{j=1}^N |e(j) - e(j+k)| \quad (3)$$

Where:

- N is the length of the frame
- $e(j)$ is the current sample
- $e(j+k)$ is the lagged sample
- k is the lag in samples

The final line searches for the lag k within the range of [25, 80] that gives the minimum AMDF value. This range corresponds to pitch periods between 400 Hz and 1250 Hz. This constraint helps avoid erroneous detections due to very small lags and ensures that only realistic pitch values are considered.

5 GenerateExcitationSignal

```

1 if voicedIdx == 1
2     if pitch <= 0
3         error('Pitch must be a positive integer.');
```

```

4     end
5     impulsePeriod = round(pitch);
6     if impulsePeriod > winLen
7         error('Pitch period exceeds frame length.');
```

```

8     end
9     for i = 1:impulsePeriod:winLen
10         excite(i) = gains;
11     end
12 else
13     excite = gains * randn(winLen, 1);
14 end
```

In LPC , accurate pitch estimation enables the generation of periodic excitation signals for voiced frames. The excitation signal is a critical component in this model because it represents the source of sound production. Depending on whether a speech frame is voiced or unvoiced, the excitation signal must simulate either a quasi-periodic train of impulses or a random noise signal. The function `generateexcitationsignal` is designed to generate such an excitation signal tailored to the characteristics of the input speech frame.

5.1 Input Arguments:

- **voicedIdx**: A binary indicator specifying whether the current frame is voiced (set to 1) or unvoiced (set to 0). This flag directs the function to choose the appropriate excitation model.
- **gains**: A scalar value representing the gain (or amplitude scaling factor) of the current frame. This parameter adjusts the energy of the generated excitation signal.
- **pitch**: The pitch period (in samples) for voiced frames. This value is used to determine the spacing between impulses in the pulse train.
- **winLen**: The length of the analysis window or frame. It defines the duration (in samples) of the output excitation signal.

5.2 Output:

- **excite**: A column vector of length **winLen** that contains the generated excitation signal appropriate for the frame (either a pulse train for voiced frames or random noise for unvoiced frames).

5.3 Impact on Speech Quality:

The quality of the excitation signal directly affects the overall quality and intelligibility of the synthesized speech. For voiced frames, the regularity of the impulses ensures that the reconstructed speech has a clear pitch and timbre. For unvoiced frames, the random noise contributes to the natural-sounding quality of fricative and plosive sounds. Errors in excitation synthesis can result in distorted or unnatural speech.

6 Conclusion

In this assignment, we implemented the LPC-10 speech synthesis algorithm in MATLAB, simulating the techniques used in the classic "Speak and Spell" device. The project guided us through the process of encoding and decoding speech signals using Linear Predictive Coding (LPC), focusing on efficiently calculating LPC coefficients, reconstructing the signal, and analyzing the quality of the reconstructed speech.

We followed a modular approach, breaking down the task into manageable functions. The first part of the implementation was dedicated to extracting LPC coefficients from speech frames. For voiced frames, we used a higher prediction order ($P=10$), while for unvoiced frames, a lower prediction order ($P=4$) was used. This flexibility helped optimize the representation of both types of speech, aligning with the variable nature of human speech.

The encoder successfully extracts key parameters, such as pitch and gain, which are then used in the decoder to reconstruct the original signal. To enhance the accuracy and quality of the speech synthesis, we implemented a series of pre- and post-processing steps, including a pre-emphasis filter, pitch detection, and overlap-and-add processing for the synthesis. In terms of analysis, we generated and visualized key metrics for the synthesized speech, including the spectrum of each frame, the prediction error in both time and frequency domains, and the magnitude of the shaping filter.

The decoder successfully reconstructed the speech signal from the encoded LPC parameters, demonstrating the effectiveness of the LPC model in capturing the essential characteristics of the speech signal while maintaining efficiency. Through the modularity of the code and careful design choices, the implementation provides a robust and insightful look into how LPC-10 coding can be used for speech synthesis, similar to the approach utilized in early speech synthesis devices. Overall, the system, designed for didactic purposes, serves as a comprehensive example of how LPC can be effectively utilized in speech compression and synthesis applications.